

HW2 — Traditional Software Design vs Function-as-a-Service

System scenario: Hotel Management System | **Language:** C++ (-O2 -std=c++17)

Student: Sari Mansour — 322449539

Both architectures implement the **same 11 operations** over the same data model

(Guest , Room , Reservation , Payment , CleaningTask , MaintenanceRequest).

#	Operation	Description
1	add_guest	Register a guest by ID
2	add_room	Add a room (type, price, floor)
3	create_reservation	Reserve a room for a guest, rejecting the request only if an existing active reservation for that room overlaps the requested date range (not merely if the room's current status is non-AVAILABLE)
4	cancel_reservation	Cancel a non-checked-in reservation
5	check_in	Check guest in -> room OCCUPIED
6	check_out	Check guest out -> room CLEANING
7	process_payment	Record a payment for a reservation
8	assign_cleaning_task	Assign a cleaning task to a room in CLEANING status; the room remains CLEANING until the task is completed
9	complete_cleaning_task	Mark a previously assigned cleaning task as finished -> room AVAILABLE
10	report_maintenance	Log maintenance; HIGH/EMERGENCY takes room offline
11	display_available_rooms	Report available room count

Operations 8 and 9 were originally a single `assign_cleaning_task` step that set the room directly to AVAILABLE, which incorrectly implied a task is completed the moment it is assigned. Splitting it into `*assign*` (op 8) and `*complete*` (op 9) fixes that and models a realistic housekeeping workflow: `check_out` -> room CLEANING -> `assign_cleaning_task` (still CLEANING) -> `complete_cleaning_task` -> room AVAILABLE.

Part 1 — Traditional Architecture

A single `HotelSystem` **class owns all data internally** through private `std::map` members; operations are **methods** that directly read and mutate this shared state.

<pre>class HotelSystem { private: map<int, Guest> guests_; map<int, Room> rooms_; map<int, Reservation> reservations_; ... payments_, cleaningTasks_, maintenanceRequests_</pre>	Files: <pre>models.h (data structs) hotel_system.h (class declaration) hotel_system.cpp (method bodies) main.cpp (demo + benchmark) feature_extension.cpp (Part 3)</pre>
--	--

Key property: direct entity lookups by ID use `std::map::find()` and therefore take **O(log n)**. Operations that search across multiple rooms or reservations (e.g. finding a replacement room, listing available rooms) require an additional linear traversal over the relevant collection and are not O(log n). State is fully encapsulated with no external visibility.

Part 2 — Function-as-a-Service Architecture

The class is replaced by **11 independent, stateless functions (handlers)**. There are no private members; data lives in an **external** `HotelStorage` struct (a `std::map`, same indexed store as Traditional) passed by reference. Each handler receives a generic **event payload** — a `std::map<string, string>`, modeling the JSON body a real cloud platform (AWS Lambda / Azure Functions) delivers to a handler — parses the fields it needs, validates, performs one task, and returns a serialized result. Invocation goes through a `FaaSOrchestrator` that looks handlers up **by name** and calls them indirectly via `std::function`, mirroring how an API Gateway routes an event to the registered handler.

```

FaaSOrchestrator (name -> handler registry)
    | invoke("check_in", event, storage)
    v
HotelStorage (external "database", std::map)
    ^ passed by reference
+-----+
add_guest(e,s)  check_in(e,s)  check_out(e,s)  ... 11 handlers
    ^              ^              ^
EventData{"id":"1", EventData{"reservationId":"1"} ...
"name":"Alice",...}

Files: models.h, storage.h (HotelStorage), faas_functions.h (EventData/ResultData,
FaaSOrchestrator, handler decls), faas_functions.cpp (11 handlers + orchestrator),
main.cpp, feature_extension.cpp

```

How the two architectures differ

Property	Traditional	FaaS
Data ownership	HotelSystem owns it (private)	External HotelStorage passed in
Storage lookup	map::find() — O(log n)	map::find() — O(log n) (same store)
Invocation	direct method call	name lookup + std::function dispatch
Argument passing	typed C++ parameters	generic string payload (marshal/parse)
State ownership	object retains state between calls	handler owns no state; all required state comes through HotelStorage
Adding a feature	modify the class	add a function; existing code untouched

Both use the same indexed storage, so the benchmark isolates the cost of the **FaaS invocation model** (marshaling + routing) rather than a data-structure difference.

Simulation scope. All handlers execute inside a single local process for benchmarking purposes. This implementation simulates the FaaS *programming model* (stateless handlers, event payloads, name-based dispatch) but does not reproduce real cloud factors such as network latency, cold starts, remote storage access, cross-process isolation, or scheduler placement. Conclusions below are scoped to the invocation model only, not to real cloud deployment performance.

Part 3 — Feature Extension: Emergency Maintenance + Auto Room Reassignment

New feature: when an emergency maintenance issue is reported, the affected room is taken offline, any active reservation is moved to a free room of the same type, and the guest is notified.

```

report emergency -> room OFFLINE -> find affected reservation
-> find replacement room (same type) -> reassign -> notify guest

```

Traditional — must modify existing files. A new method `emergencyMaintenance()` was added to `hotel_system.h` and `hotel_system.cpp`. One method now touches **three private members** (`rooms_`, `reservations_`, `guests_`) and mixes five responsibilities (report, search reservations, search rooms, reassign, notify). This increases coupling — the class grows with every cross-cutting feature and the change risks breaking existing behaviour because it edits shared code.

FaaS — modeled as a new FaaS workflow, not four independent handlers. The same feature was implemented entirely in `feature_extension.cpp`: one externally invoked chain function (`emergency_chain`) coordinates four small, stateless helper functions (`find_affected_reservations`, `find_replacement_room`, `reassign_reservation`, `notify_guest`) over the external `HotelStorage`. This is closer to a FaaS orchestration workflow containing helper functions than four independently triggered, independently deployable FaaS services — the four helpers are not registered in the `FaaSOrchestrator` and are not individually invocable by name. The file builds its **own local FaaSOrchestrator instance** and registers only the existing handlers it reuses (`add_guest`, `add_room`, `create_reservation`, `check_in`, `report_maintenance`, `display_available_rooms`) — the main system's orchestrator setup in `main.cpp` is not

touched, so no existing handler registration was modified. Because no existing handlers were modified, ****the risk of introducing regressions into existing operations is lower**** — but not zero: the new chain writes into the same shared `HotelStorage` (reservations, rooms) that every other handler reads, so an integration bug in `reassign_reservation` (for example) could still corrupt state that `check_in` or `process_payment` later rely on.

Scope limitation. `emergencyMaintenance()` / `emergency_chain()` reassigns the `roomId` of any matching **active** reservation, regardless of whether that guest has already checked in. For a genuinely **future** reservation this is sufficient. For a guest who is **already checked in**, updating the stored room number alone does not perform an actual physical relocation (new key, moved luggage, updated occupancy status) — that case is out of scope for this simplified, partial Part 3 implementation and is not specially handled.

	Traditional	FaaS
Existing application source files modified (<code>hotel_system.*</code> / <code>faas_functions.*</code>)	2	0
Where the new logic lives	added into <code>hotel_system.h</code> / <code>hotel_system.cpp</code> (grows the core class)	added entirely within <code>feature_extension.cpp</code> (separate file; core handlers untouched)
Build/script changes required	0 (extension binary was already wired into <code>script.sh</code>)	0 (extension binary was already wired into <code>script.sh</code>)
Regression risk	Medium (shared class edited directly)	Lower, but not zero (shared storage still reused)
Debugging the full flow	Easy (one call stack)	Harder (multi-step, no atomicity)

Verdict. For this specific emergency-reassignment feature, the FaaS design required fewer modifications to existing components and was easier to extend, because it could add the new behaviour without touching a single existing handler or the main orchestrator registration. The Traditional design remained easier to trace and manage, because the whole flow executes as one synchronous method call within a single call stack instead of a multi-step chain over shared storage — this is simpler consistency management in this single-process implementation, not a transactional guarantee.

Part 4 — Performance Evaluation

Workload (identical for both, benchmark mode, ~76,000 operations):

10,000 guests, 1,000 rooms, 10,000 reservations, 10,000 check-ins, 10,000 payments, 10,000 check-outs, 10,000 cleaning-task assignments, 10,000 cleaning-task completions, 5,000 maintenance reports.

Measured on the course KVM (Ubuntu, `perf stat -r 5, /usr/bin/time`). Both implementations use the same indexed `std::map` storage, so this experiment isolates the cost of the **FaaS invocation model** (event marshaling + name-based dispatch) rather than a data-structure artifact. All metrics below are the **mean of 5 runs** as reported by `perf stat -r 5` .

Methodology note. Because a shared KVM guest can expose only a few hardware PMU counter slots, hardware events were split into three separate `perf stat -r 5` calls per binary (cycles/instructions/context-switches; branches/branch-misses/cache-references/cache-misses; page-faults/minor-faults/major-faults/cpu-migrations) to avoid counter eviction. An earlier version of this benchmark showed `cycles` reading exactly 0 when all events were requested in a single call — that measurement was discarded and re-collected this way. A separate issue was also caught by profiling itself: the first version of the date-overlap check (Part 2/3) scanned **every reservation ever created** instead of only the reservations for the specific room being booked, which inflated instruction counts by ~20x for both architectures equally and masked the real architectural gap. That was fixed with a per-room reservation index (`roomReservationIndex`) before this final measurement was taken. The final numbers below were reproduced across two independent runs within ~1%, confirming they are stable and not measurement noise.

Metric	Traditional (mean, 5 runs)	FaaS (mean, 5 runs)	Result
Execution time	0.0253 s	0.0645 s	FaaS 2.6x slower
CPU cycles	55.1 M	151.8 M	FaaS 2.8x more
Instructions	71.4 M	322.0 M	FaaS 4.5x more
Instructions/cycle	1.28	2.12	FaaS higher despite more instructions
Branches	16.8 M	76.0 M	FaaS 4.5x more (tracks instruction count)
Branch-misses	604,043 (3.59% of branches)	520,858 (0.68% of branches)	FaaS mispredicts far less per branch
Cache-references	186,737	227,680	FaaS +22%
Cache-misses	67 misses	104 misses	both very low; see caveat below
Page-faults (100% minor, 0 major, both)	1,343	1,349	nearly identical (+0.4%)
CPU-migrations	0	0	both pinned to one core for the whole run
Context-switches	5	7	FaaS more (longer runtime)
Max memory (RSS)	8,340 KB	8,832 KB	FaaS +5.9%

Which performed better and why. Traditional is faster, by about **2.6x** in wall time and **2.8x** in CPU cycles. Because both versions use identical `std::map` storage, none of this gap comes from the data structure — it is the cost of the `**FaaS invocation model**`:

- **Event marshaling and allocation.** Every call constructs a node-based `std::map<string, string>`, which allocates a tree node plus heap storage for each key and value string, then converts each argument with `std::to_string`; every handler parses it back with `std::stoi` / `std::stod`. This happens for every one of the ~76,000 operations and is the most likely dominant cost — it is consistent with why FaaS needs **4.5x more instructions** (and proportionally 4.5x more branches) for the same logical work.
- **Tested and ruled out: page faults are not the source of FaaS's overhead.** The hypothesis that FaaS's heavier per-call allocation would trigger more OS-level page faults was directly tested and **not supported**: page-faults are nearly identical between the two (1,343 vs 1,349, all minor, zero major faults, zero CPU-migrations for both — both processes stayed pinned to one core the whole run). This indicates the allocator satisfies FaaS's many small, transient `map` / `string` allocations by reusing memory already obtained from the OS (heap arena/free-list reuse) rather than requesting new pages per call. The extra instructions therefore come from allocator bookkeeping, string construction/copy, and parsing work happening **within** already-mapped memory — not from additional virtual-memory activity.
- **Name-based dispatch.** `FaaSOrchestrator::invoke()` performs a `std::string` key lookup in a `std::map<string, std::function<...>>` and then an indirect call through `std::function`, versus a direct (often inlined) method call in Traditional.
- **Higher IPC (2.12 vs 1.28) despite more instructions.** With branch data now collected, FaaS's branch-misprediction rate (0.68%) is more than 5x lower than Traditional's (3.59%), even though FaaS executes 4.5x more branches in absolute terms. This is consistent with (though does not on its own fully prove) the higher IPC: FaaS's marshaling code is largely straight-line string parsing with simple, well-predicted branches, while Traditional's map-based logic (tree traversals, the per-room overlap check) has proportionally more data-dependent branches. A complete causal explanation would still benefit from frontend/backend stall-cycle breakdowns, which were not collected here.
- **Cache-misses.** The raw `perf` output was checked and confirmed to report the actual `cache-misses` event (not a different cache-level event, and with no `<not supported>` / `<not counted>` markers) — the counters simply produced genuinely very low, noisy absolute values (67 vs 104, with variance up to ±980% across the 5 runs) relative to tens of millions of instructions. Because of that variance, generic hardware cache counters are reported here for completeness but no conclusion — such as "the working set fits in cache" — is drawn from this metric, and it is not used to support the final conclusion.
- **Memory is nearly identical (+5.9% for FaaS)** — consistent with the page-fault finding above, this small increase is the small, transient event/result maps, which are freed

immediately after each call and mostly satisfied from already-allocated heap pages.

Interpretation. This implementation simulates the FaaS programming model inside one local process. It measures the overhead of event construction, string parsing, handler lookup, and indirect invocation. It does not measure real cloud factors such as network latency, cold starts, remote storage, or process-level isolation. A real cloud deployment would introduce additional factors such as network latency, serialization, remote storage access, and cold starts. Therefore, the absolute performance results of this local simulation should not be interpreted as predictions of cloud execution time — they quantify only the marshaling-and-dispatch tax of the FaaS programming model itself.

Conclusion

The two implementations provide the same hotel-management functionality but use substantially different structures. In this local, single-process experiment, the Traditional implementation performed better, completing the workload approximately 2.6 times faster and using 2.8 times fewer CPU cycles. The main reason is that Traditional operations use typed, direct method calls, while every simulated FaaS invocation constructs string-based event payloads, performs conversions and allocations, looks up a handler by name, and invokes it indirectly.

For the emergency room-reassignment feature, the FaaS design required no modifications to existing application handlers and allowed the new workflow to be implemented in a separate source file. The Traditional design required changes to the central class but remained easier to trace and manage synchronously through one call stack. Therefore, for this particular feature, the FaaS structure provided easier extension with lower regression risk, while the Traditional structure provided simpler debugging and consistency management.

The FaaS programming model is designed to enable isolation and independent scaling in a real deployment, although those properties are not implemented or measured by this local simulation. This experiment evaluates only a local simulation of the FaaS programming model — it does not measure real cloud properties such as cold starts, network communication, remote databases, process isolation, or automatic scaling. The results therefore quantify only the local cost of marshaling and dispatch, and the reduced number of existing components changed for the selected extension — not the total performance of a deployed cloud FaaS system.

Appendix — AI Tool Usage

An AI assistant (GitHub Copilot) was used only for **architectural discussion and code organization**: brainstorming the Part-3 room-reassignment feature, structuring the code into the multi-file layout, and helping interpret the `perf` output. The scenario choice, benchmark design, final code, and all written analysis were decided and verified by the student. No external repository implementation was copied.